
Part B

Ex: 10

Title: Optimal Product Selection for a Limited Shelf Space

Problem Description: You are a store manager responsible for stocking products on the shelves. The store has limited shelf space, and you want to maximize the total profit by selecting the most valuable products to display. Each product has a specific weight and profit associated with it.

Method: Solve the 0/1 Knapsack Problem using dynamic programming or other optimization techniques. The key idea is to build a table that stores the maximum value achievable for different weights and items.

Theory Reference: Module 4

Algorithm

ALGORITHM DPKnapsack(n,m,p,w)

//function: solves knapsack problem using dynamic programming

//Input: A nonnegative integer 'n' and 'm', where n indicates the number of items and m the knapsack capacity with their profits & weight.

//Output: Optimal profit and items selected.

for i←0 to n do

 for j←0 to m do

 if i=0 or j=0 then

$V[i,j] \leftarrow 0$

 else if $j < W[i]$ then

$V[i,j] \leftarrow V[i-1,j]$

 else

$V[i,j] \leftarrow \max(V[i-1,j], p[i] + V[i-1, j - W[i]])$

return V

Program

```
import java.util.Scanner;
```

```
public class DPKnapsack
{
    static int n,m,p[],w[];

    static void knapsackDP()
    {
        int[][] v = new int[n+1][m+1];

        for(int i=0;i<=n;i++)
        {
            for(int j=0;j<=m;j++)
            {
                if(i==0 || j==0)
                    v[i][j]=0;

                else if(j<w[i])//weight of the item is larger than capacity
                    v[i][j]=v[i-1][j];

                else
                    v[i][j]=max(v[i-1][j],p[i]+v[i-1][j-w[i]]);//maximization
            }
        }

        for(int i=0;i<=n;i++)
        {
            for(int j=0;j<=m;j++)

                System.out.print(v[i][j]+" ");
        }
    }
}
```

```
        System.out.println();
    }

    /* tracking back the optimal solution vector */

    System.out.println("OPTIMAL PROFIT IS:"+v[n][m]);

    System.out.print("Products selected for shelf that yields maximum profit are:");

    while(n!=0)
    {
        if(v[n][m] != v[n-1][m])
        {
            System.out.print(n + " ");

            m = m - w[n];
        }

        n--;
    }
}

static int max(int a, int b)
{
    return(a>b?a:b);
}

public static void main(String[] args)
{
```

```
Scanner s = new Scanner(System.in);

System.out.println("Enter the no. of products");

n = s.nextInt();

p=new int[n+1];

w=new int[n+1];

System.out.println("Enter the weights of n products");

for(int i=1;i<=n;i++)

    w[i]=s.nextInt();

System.out.println("Enter the profits of n products");

for(int i=1;i<=n;i++)

    p[i]=s.nextInt();

System.out.println("Enter the capacity of shelf(Knapsack)");

m = s.nextInt();

knapsackDP();

}

}
```

Output

```
run:
Enter the no. of products
4
Enter the weights of n products
2 1 3 2
Enter the profits of n products
12 10 20 15
Enter the capacity of shelf(Knapsack)
5
0 0 0 0 0 0
0 0 12 12 12 12
0 10 12 22 22 22
0 10 12 22 30 32
0 10 15 25 30 37
OPTIMAL PROFIT IS:37
Products selected for shelf that yields maximum profit are:4 2 1 BUILD SUCCESSFUL (total time: 16 seconds)
```